Sensitive Data Access Control System Documentation

Overview

This project implements a system that controls user access to sensitive data based on predefined rules. Users can only access non-sensitive data or both sensitive and non-sensitive data depending on their access level. The program reads data from CSV files and dynamically determines which data columns users can access based on sensitivity.

Folder Structure

The project folder should have the following structure:

bash
Copy code

```
master/
│
├── file_a.csv    # Data file (contains records with sensitive and non-sensitive data)
├── file_b.csv    # Column sensitivity file (marks columns as sensitive or non-sensitive)
├── file_c.csv    # User access file (defines whether users have access to sensitive data)
└── main.py       # Python script that runs the access control system
```

Files and Their Purpose

1. file_a.csv

This file contains the raw data that users are trying to access. The data includes both sensitive and non-sensitive fields.

Structure:

- **Columns**: `RecID, fname, lname, mname, address, city, state, zip, ssn`
- Each row represents a record, with fields that may or may not be sensitive.

Example:

csv
Copy code

```
RecID,fname,lname,mname,address,city,state,zip,ssn
1,John,Doe,A,123 Main St,Anytown,CA,90210,123-45-6789
2,Jane,Smith,B,456 Maple St,Othertown,NY,10001,987-65-4321
3,Jim,Brown,C,789 Oak St,Smallville,TX,75001,111-22-3333
```

2. file_b.csv

This file defines which columns in `file_a.csv` are sensitive or non-sensitive. The sensitivity information is critical for determining what data users can access.

Structure:

- **Columns**:
  - `column_name`: Name of the columns in `file_a.csv`.
  - `sensitivity`: Whether the column is marked as sensitive (`yes`) or non-sensitive (`no`).

Example:

csv
Copy code

```
column_name,sensitivity
RecID,no
fname,no
lname,no
mname,no
address,yes
city,no
state,no
zip,no
ssn,yes
```

3. file_c.csv

This file contains the list of users and whether they have access to sensitive data. Users who do not have access to sensitive data will only be able to view non-sensitive columns.

Structure:

- **Columns**:
  - `username`: The user's name.
  - `access_sensitive`: Indicates whether the user has access to sensitive data (`yes` or `no`).

Example:

csv

Copy code

```
username,access_sensitive
user1,yes
user2,no
user3,no
```

Python Script (main.py)

The `main.py` file is the core of the system, where the logic for validating users, checking access levels, and displaying data is implemented.

How it Works:

Importing Libraries: The script uses `pandas` to load and manipulate the CSV files.

python

Copy code

```
import pandas as pd
```

1.
2. **Loading the Files**:
   - `file_a.csv`: The data file.
   - `file_b.csv`: The sensitivity file.
   - `file_c.csv`: The user access file.

python

Copy code

```
file_a = pd.read_csv('file_a.csv')
file_b = pd.read_csv('file_b.csv')
file_c = pd.read_csv('file_c.csv')
```

3.
4. **User Validation**:
    - A function `validate_user` checks if the username exists in `file_c.csv` and whether they have access to sensitive data.

python
Copy code
```
def validate_user(username):
    user_info = file_c[file_c['username'] == username]
    if user_info.empty:
        return None, False
    else:
        has_access_to_sensitive =
user_info.iloc[0]['access_sensitive'] == 'yes'
        return user_info, has_access_to_sensitive
```

5.
6. **Fetching Accessible Columns**:
    - Based on the sensitivity settings in `file_b.csv` and the user's access, the function `get_accessible_columns` determines which columns the user can view.

python
Copy code
```
def get_accessible_columns(has_access_to_sensitive):
    if has_access_to_sensitive:
        accessible_columns = file_b['column_name'].tolist()
    else:
        accessible_columns = file_b[file_b['sensitivity'] ==
'no']['column_name'].tolist()
    return accessible_columns
```

7.
8. **Displaying Options to the User**:
    - The script shows the accessible columns to the user and prompts them to select the columns they want to view.

python
Copy code
```
def display_options_to_user(accessible_columns):
    print(f"Accessible columns: {', '.join(accessible_columns)}")
```

```
    selected_columns = input("Enter the columns you want to view,
separated by commas: ").split(',')
    selected_columns = [col.strip() for col in selected_columns if
col.strip() in accessible_columns]
    return selected_columns
```

9.
10. **Displaying Selected Data**:
    - After the user selects columns, the function `display_selected_data` displays the corresponding data from `file_a.csv`.

python
Copy code

```
def display_selected_data(selected_columns):
    if selected_columns:
        print(file_a[selected_columns])
    else:
        print("No valid columns selected.")
```

11.
12. **Main Flow**:
    - The main function ties everything together: it validates the user, retrieves accessible columns, and displays the data.

python
Copy code

```
def main():
    username = input("Enter your username: ")
    user_info, has_access = validate_user(username)
    if user_info is None:
        print("User does not exist or is not allowed.")
        return
    accessible_columns = get_accessible_columns(has_access)
    selected_columns = display_options_to_user(accessible_columns)
    display_selected_data(selected_columns)

if __name__ == "__main__":
    main()
```

13.

Running the Program

Step-by-Step Guide:

1. **Create the folder structure**:
    - Inside the `master` folder, create the files `file_a.csv`, `file_b.csv`, and `file_c.csv`.
2. **Populate the CSV files** with the content provided earlier in this document.
3. **Create the `main.py` file** inside the `master` folder and copy the Python code into it.
4. **Install necessary dependencies** (if needed):

If you don't have `pandas` installed, run:
Copy code
```
pip install pandas
```

    -
5. **Run the program**:

In your terminal or command prompt, navigate to the `master` folder:
bash
Copy code
```
cd path/to/master
```

    -

Run the script:
css
Copy code
```
python main.py
```

    -
6. **Program Flow**:
    - The script will prompt the user to enter their username.
    - Based on their access level, it will display the columns they are allowed to view.
    - The user can then select specific columns, and the corresponding data will be displayed.

Conclusion

This system allows users to securely access data based on predefined sensitivity rules. It ensures that sensitive data is only accessible to authorized users, while non-sensitive data is

available to others. This is achieved by dynamically checking user permissions and filtering the data accordingly.

Let me know if you have any questions or need further clarifications!